

Evidencias de Conceptos - POO

Taller CRUD Tienda (Programacion Orientada a Objetos)

Estudiante: _____ / Fecha: _____

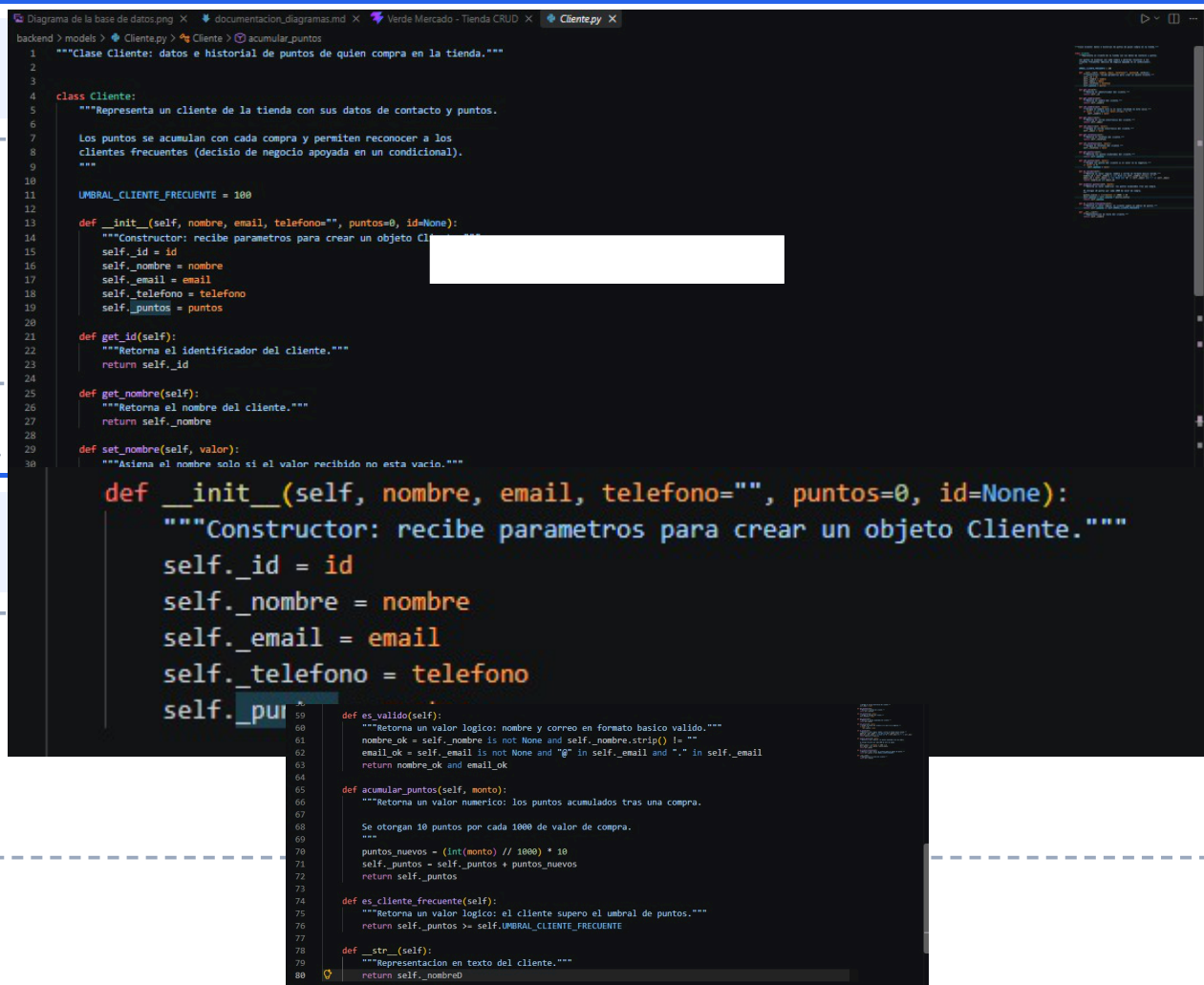
Como completar este documento: para cada uno de los nueve conceptos del enunciado se indica que se demuestra en el código. Pegar en el recuadro punteado la captura de pantalla correspondiente (consola, código fuente o pantalla del aplicativo) que respalde la evidencia.

1. Clase

Se evidencia en `backend/models/*.py`: `class Categoria`, `class Producto`, `class Cliente`, `class Pedido` y `class DetallePedido`.

2. Atributos

3.



```
Diagrama de la base de datos.png X documentacion_diagramas.md X Verde Mercado - Tienda CRUD X Cliente.py X
backend > models > Cliente.py > Cliente > acumular_puntos
1 """Clase Cliente: datos e historial de puntos de quien compra en la tienda."""
2
3
4 class Cliente:
5     """Representa un cliente de la tienda con sus datos de contacto y puntos.
6
7     Los puntos se acumulan con cada compra y permiten reconocer a los
8     clientes frecuentes (decisión de negocio apoyada en un condicional).
9     """
10
11     UMBRAL_CLIENTE_FRECUENTE = 100
12
13     def __init__(self, nombre, email, telefono="", puntos=0, id=None):
14         """Constructor: recibe parametros para crear un objeto Cliente.
15
16         self.id = id
17         self.nombre = nombre
18         self.email = email
19         self.telefono = telefono
20         self.puntos = puntos
21
22     def get_id(self):
23         """Retorna el identificador del cliente."""
24         return self._id
25
26     def get_nombre(self):
27         """Retorna el nombre del cliente."""
28         return self._nombre
29
30     def set_nombre(self, valor):
31         """Asigna el nombre solo si el valor recibido no esta vacio."""
32
33     def __init__(self, nombre, email, telefono="", puntos=0, id=None):
34         """Constructor: recibe parametros para crear un objeto Cliente."""
35         self._id = id
36         self._nombre = nombre
37         self._email = email
38         self._telefono = telefono
39         self._puntos = puntos
40
41     def es_valido(self):
42         """Retorna un valor logico: nombre y correo en formato basico valido."""
43         nombre_ok = self._nombre is not None and self._nombre.strip() != ""
44         email_ok = self._email is not None and "@" in self._email and "." in self._email
45         return nombre_ok and email_ok
46
47     def acumular_puntos(self, monto):
48         """Retorna un valor numerico: los puntos acumulados tras una compra.
49
50         Se otorgan 10 puntos por cada 1000 de valor de compra.
51         """
52         puntos_nuevos = (int(monto) // 1000) * 10
53         self._puntos = self._puntos + puntos_nuevos
54         return self._puntos
55
56     def es_cliente_frecuente(self):
57         """Retorna un valor logico: el cliente supero el umbral de puntos."""
58         return self._puntos >= self.UMBRAL_CLIENTE_FRECUENTE
59
60     def __str__(self):
61         """Representacion en texto del cliente."""
62         return self._nombre
```

4. Constructor

Se evidencia en `__init__` de las cinco clases, con parametros que inicializan los atributos del objeto.

5. Paso de parametros

Se evidencia en el recorrido del dato: formulario (Vista) → JSON → ruta → controlador → modelo; y en metodos como `agregar_producto(producto, cantidad)`.

6. Retorno de valores numericos y logicos

```
16 def crear(self, nombre, descripcion, precio_compra, precio_venta, stock,
17         categoria_id=None, imagen=""):
18     """Crea un producto tras validar datos, precios y categoria.
19
20     Retorna el objeto Producto creado (con su id de base de datos)
21     o None si algun dato es invalido.
22     """
23     if nombre is None or nombre.strip() == "":
24         return None
25     if precio_venta <= 0 or precio_compra < 0 or stock < 0:
26         return None
27
28     categoria = None
29     if categoria_id is not None:
30         categoria = self._repositorio.obtener_categoria(categoria_id)
31         if categoria is None:
32             return None
33
34     producto = Producto(
35         nombre=nombre,
36
37 def calcular_total(self, pedido_id):
38     """Retorna el total del pedido o None si el pedido no existe."""
39     pedido = self.obtener_por_id(pedido_id)
40     if pedido is None:
41         return None
42     return pedido.calcular_total()
43
44     return None
45     return self._repositio
```

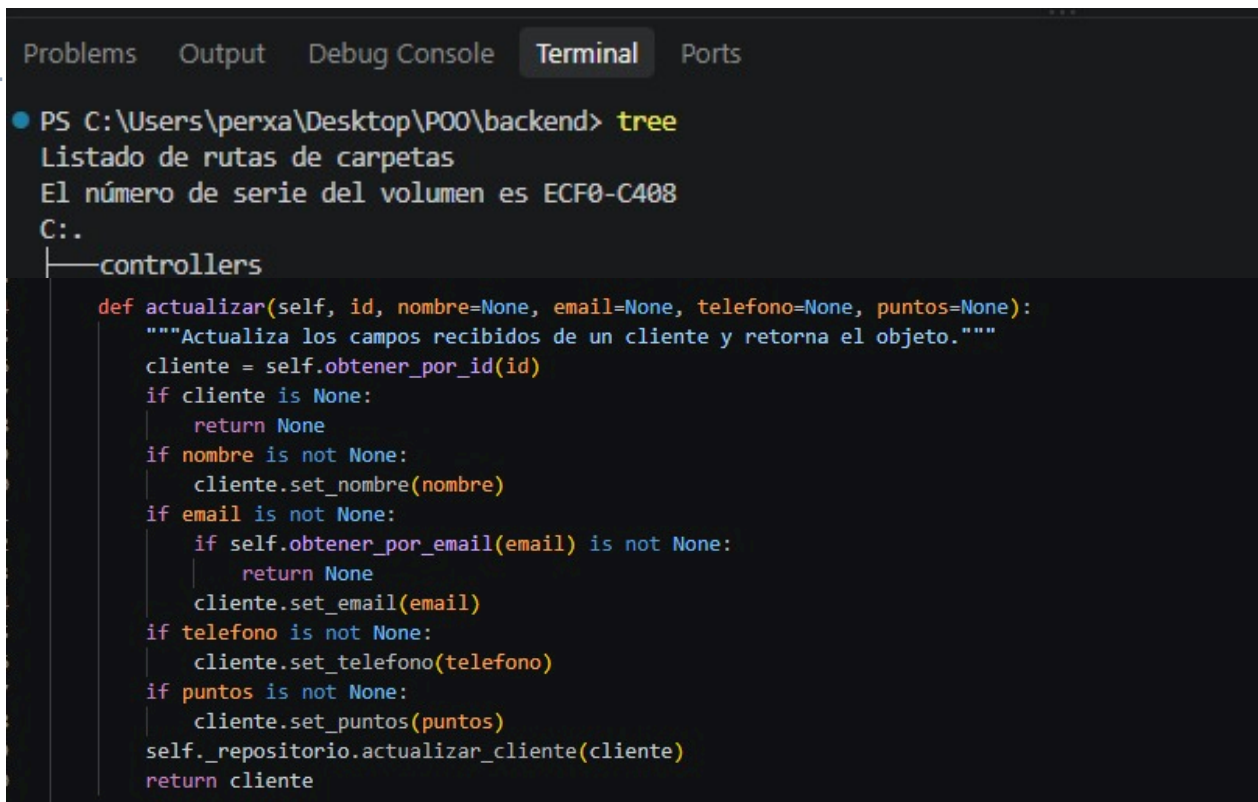
```
4 class DetallePedido:
5     """Representa la compra de un producto especifico en cantidad determinada.
6
7     Usa el Pedido con el producto: cada detalle guarda la referencia al objeto
8     Producto y el precio unitario con el que se compro en ese momento.
9     """
10
11     def __init__(self, producto, cantidad, precio_unitario=None, id=None):
12         """Constructor: recibe un objeto Producto y la cantidad a comprar.
13
14         Si no se indica precio_unitario, se toma el precio de venta actual
15         del producto (uso de objetos: se lee un dato de otro objeto).
16         """
17         self.id = id
18         self_producto = producto
19         self_cantidad = cantidad
20         if precio_unitario is None:
21             precio_unitario = producto.get_precio_venta()
22         self_precio_unitario = precio_unitario
23
24     def get_id(self):
25         """Retorna el identificador del detalle."""
26         return self.id
27
28     def get_producto(self):
29         """Retorna el objeto Producto asociado a este detalle."""
30         return self_producto
31
32     def get_cantidad(self):
33         """Retorna la cantidad de unidades compradas de este producto.
34         """
35         return self_cantidad
```

8. Arquitectura MVC

Se evidencia en la separación por capas: React (Vista) → API REST/controladores `backend/routes` y `backend/controllers` → modelos `backend/models` → persistencia SQLite `backend/data`.

9. Condicionales

Se evidencia en las validaciones de los controladores (correo, precios, stock, estados) y en el render condicional de React (por ejemplo, mostrar el formulario solo cuando el pedido está pendiente).



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\perxa\Desktop\P00\backend> tree
Listado de rutas de carpetas
El número de serie del volumen es ECF0-C408
C:..
|---controllers

def actualizar(self, id, nombre=None, email=None, telefono=None, puntos=None):
    """Actualiza los campos recibidos de un cliente y retorna el objeto."""
    cliente = self.obtener_por_id(id)
    if cliente is None:
        return None
    if nombre is not None:
        cliente.set_nombre(nombre)
    if email is not None:
        if self.obtener_por_email(email) is not None:
            return None
        cliente.set_email(email)
    if telefono is not None:
        cliente.set_telefono(telefono)
    if puntos is not None:
        cliente.set_puntos(puntos)
    self._repositorio.actualizar_cliente(cliente)
    return cliente
```